

Αρχές Γλωσσών Προγραμματισμού και Μεταφραστών



UNIVERSITY OF
PATRAS
ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ

Python Project

Όνομα: Αγγελόπουλος Γρηγόρης-Παναγιώτης
(AM: 1115514, Email: up1115514@ac.upatras.gr, Έτος: 2ο)

Όνομα: Κοπίτσας Νικόλαος
(AM: 1115515, Email: up1115515@ac.upatras.gr, Έτος: 2ο)

Περιεχόμενα

1. Σύνοψη και Ανάλυση Εργαλείων Υλοποίησης	2
<i>(Python, Requests, BeautifulSoup, Pandas, Tkinter, Matplotlib)</i>	
2. Σύστημα Συλλογής και Διαχείρισης Δεδομένων	3
2.1 Πηγές Δεδομένων (Ανάλυση Web Scraping & API)	3
2.2 Δομή και Αποθήκευση σε Αρχείο CSV	5
2.3 Μηχανισμός Κανονικοποίησης και Ομογενοποίησης	6
3. Σχεδιασμός και Υλοποίηση Γραφικού Περιβάλλοντος (GUI)	8
3.1 Αρχιτεκτονική Διεπαφής με Tkinter	8
3.2 Λειτουργίες Αναζήτησης και Φιλτραρίσματος	8
3.3 Ενσωμάτωση Μηνυμάτων Προόδου και Ελέγχου	8
4. Στατιστική Επεξεργασία και Οπτικοποίηση	9
4.1 Ανάλυση Δεδομένων με Pandas	9
4.2 Γραφική Απεικόνιση (Bar, Pie, Line Plot)	9
4.3 Ερμηνεία και Τάσεις Δεδομένων	9
5. Μηχανή Έξυπνων Συστάσεων (Recommendation Engine)	10
5.1 Λογική και Σχεδιασμός του Αλγορίθμου Κατάταξης	10
5.2 Υπολογισμός Σύνθετης Βαθμολογίας (Composite Score)	10
5.3 Διαχείριση Ελλιπών Στοιχείων και Παρουσίαση	10
6. Τεκμηρίωση Υλοποίησης (Screenshots & Λύσεις)	12
6.1 Παραδοχές Σχεδιασμού	12
6.2 Προβλήματα που Αντιμετωπίστηκαν και Λύσεις	12
6.3 Στιγμιότυπα Εκτέλεσης (Screenshots)	13
7. Παράρτημα: Κώδικας Python	14
7.1 Τμηματική Παρουσίαση Κώδικα ανά Ζητούμενο	14

1. Σύνοψη και Ανάλυση Εργαλείων Υλοποίησης

Για την υλοποίηση της εφαρμογής χρησιμοποιήθηκε η γλώσσα προγραμματισμού **Python**, η οποία επιλέχθηκε λόγω της ευελιξίας της και της πληθώρας βιβλιοθηκών που προσφέρει για τη διαχείριση δεδομένων. Η αρχιτεκτονική της εφαρμογής βασίζεται σε τέσσερις βασικές Βιβλιοθήκες:

1. Requests & BeautifulSoup (Συλλογή Δεδομένων)

Για τη συλλογή των δεδομένων από το διαδίκτυο (Web Scraping) χρησιμοποιήθηκαν οι βιβλιοθήκες **requests** και **BeautifulSoup**. Η πρώτη αναλαμβάνει την επικοινωνία με τους servers και τη λήψη του περιεχομένου, ενώ η δεύτερη μας επιτρέπει να «διαβάζουμε» τον κώδικα HTML και να απομονώνουμε τις πληροφορίες που χρειαζόμαστε, όπως τίτλους μαθημάτων, διάρκειες και κόστη.

2. Pandas (Διαχείριση και Επεξεργασία)

Η βιβλιοθήκη **pandas** χρησιμοποιήθηκε για όλη τη διαχείριση του αρχείου **.csv**. Μας διευκόλυνε σημαντικά στην ανάγνωση και εγγραφή των δεδομένων (Functions: `to_csv()`, `read_csv()`), την επεξεργασία των πινάκων δεδομένων (DataFrames) και την υλοποίηση του μηχανισμού κανονικοποίησης (normalization) των κατηγοριών. Με το **pandas**, μπορέσαμε να ομογενοποιήσουμε τις διαφορετικές κατηγορίες και τα επίπεδα δυσκολίας από τις πηγές μας σε μια ενιαία μορφή.

3. Tkinter (Γραφικό Περιβάλλον)

Για την αλληλεπίδραση με τον χρήστη χρησιμοποιήθηκε το **tkinter**. Μέσω αυτού σχεδιάστηκε το GUI της εφαρμογής, το οποίο περιλαμβάνει τα φίλτρα αναζήτησης, τον πίνακα προβολής των μαθημάτων και τα πεδία εισαγωγής για τη μηχανή συστάσεων, pop up windows για έλεγχο επιλογών χρήστη, προσφέροντας ένα φιλικό περιβάλλον χωρίς την ανάγκη χρήσης τερματικού.

4. Matplotlib (Οπτικοποίηση)

Τέλος, η βιβλιοθήκη **matplotlib** χρησιμοποιήθηκε για την παραγωγή των στατιστικών γραφημάτων. Με αυτήν υλοποιήθηκαν τα τρία απαιτούμενα γραφήματα (Bar, Pie και Line plot), μετατρέποντας τα αποθηκευμένα δεδομένα του CSV σε οπτικές πληροφορίες για τη διάρκεια, το κόστος και την κατανομή δυσκολίας των μαθημάτων.

Η συνδυαστική χρήση των παραπάνω εργαλείων επέτρεψε τη δημιουργία μιας εφαρμογής, η οποία είναι ικανή να διαχειρίζεται σωστά δεδομένα, από τη συλλογή τους έως την τελική οπτικοποίηση και ανάλυση τους.

2. Σύστημα Συλλογής και Διαχείρισης Δεδομένων

2.1 Πηγές Δεδομένων (Ανάλυση 3 API και 3 Web Scraping πηγών)

Για τη συλλογή των δεδομένων χρησιμοποιήθηκαν συνολικά 4 πηγές, ώστε να έχουμε πολλά διαφορετικά μαθήματα και να καλύψουμε τις απαιτήσεις της άσκησης.

Web Scraping: Για την άντληση δεδομένων από ιστοσελίδες χρησιμοποιήσαμε τις βιβλιοθήκες `requests` και `BeautifulSoup`. Οι πηγές που υλοποιήθηκαν είναι:

- **Harvard University:** Ο κώδικας διαβάζει τα `data-attributes` των στοιχείων HTML για να πάρει απευθείας πληροφορίες όπως η τιμή, η δυσκολία και ο πάροχος κτλ.

```

1  def HarvardScraper(url):
2      all_Harvard_courses = []
3      number_of_courses = 6
4
5
6      res = requests.get(url)
7      soup = BeautifulSoup(res.text, 'html.parser')
8
9      # Find the main content container
10     content = soup.find('div', class_='main-wrapper_grid-
        container_full')
11
12     if not content:
13         print("No article content found.")
14         return pd.DataFrame(all_Harvard_courses)
15
16     blocks = content.find_all("div", class_='views-row')
17
18
19     random.shuffle(blocks)
20
21     for block in blocks:
22
23         if len(all_Harvard_courses) >= number_of_courses:
24             break
25
26         index = block.find("div", class_="datalayer-values")
27
28         if not index:
29             continue
30
31         title = index.get("data-item-name") or "Unknown"
32         price = float(index.get("data-course-price")) or np.
            nan
33         difficulty = index.get("data-course-difficulty") or "
            Unknown"
34         subject_category = index.get("data-item-category") or
            "Unknown"
35         provider = index.get("data-course-school") or "Unknown
            "
36

```

```

37     course_len = index.get("data-course-length") or "N/A"
38
39     if course_len != "N/A":
40         if course_len.lower().find("week") != -1:
41             course_len = course_len[:course_len.lower().
42                 find("week")]
43             course_length = np.ceil(7 * int(course_len)).
44                 astype(int)
45         if course_len.lower().find("month") != -1:
46             course_len = course_len[:course_len.lower().
47                 find("month")]
48             course_length = np.ceil(31 * int(course_len)).
49                 astype(int)
50         else:
51             course_length=0
52
53     course_language = index.get("data-course-language") or
54         "Unknown"
55
56     all_Harvard_courses.append({
57         "Title": title,
58         "Price_(in_$)": price,
59         "Difficulty": difficulty,
60         "Subject_Category": subject_category,
61         "Provider": provider,
62         "Course_Length_(in_Days)": course_length,
63         "Course_Language": course_language
64     })
65
66 df = pd.DataFrame(all_Harvard_courses)
67
68 return df

```

- **Class Central:** Χρησιμοποιήθηκε η σελίδα με τα δωρεάν μαθήματα. Επειδή τα metadata ήταν σε μορφή JSON μέσα στο HTML, χρησιμοποιήσαμε τη βιβλιοθήκη json για να τα εξάγουμε σωστά.

```

1
2 def ClassCentralScraper(url):
3     all_ClassCentral_courses = []
4     number_of_courses = 6
5
6
7
8     res = requests.get(url)
9     soup = BeautifulSoup(res.text, 'html.parser')
10
11     # Find the main content container
12     content = soup.find('div', class_='catalog-grid__results')
13
14     if not content:
15         print("No article content found.")
16         return pd.DataFrame(all_ClassCentral_courses)

```

```
17
18     blocks = content.find_all("li", class_='bg-white border-
        all border-gray-light padding-xsmall radius-small
        margin-bottom-small medium-up-padding-horz-large medium
        -up-padding-vert-medium course-list-course')
19
20
21     random.shuffle(blocks)
22
23     for block in blocks:
24
25         if len(all_ClassCentral_courses) >= number_of_courses:
26             break
27
28         index = block.find("p", class_="text-2 margin-bottom-
            xsmall")
29
30         if not index:
31             continue
32
33         a_tag = index.find("a", {"data-track-props": True})
34
35         if not a_tag:
36             continue
37
38         try:
39             props = json.loads(a_tag["data-track-props"])
40         except:
41             continue
42
43         title=props.get("course_name") or "Unknown"
44         course_language = props.get("course_language") or "
            Unknown"
45         difficulty = props.get("course_level") or "Unknown"
46
47
48
49         if props.get("course_is_free"):
50             price=0
51         else:
52             price=np.nan
53
54
55         subject_category = props.get("course_subject")
56
57         if props.get("course_is_university"):
58             provider = props.get("course_institution")
59         else:
60             provider = "Unknown"
61
62         index = block.find("span", class_="text-3 margin-left-
            small line-tight")
63
```

```

64     course_length = index.get_text(strip=True)
65     temp = course_length.split()
66
67     weeks = 0
68     hours = 0
69
70
71     if "week" in course_length:
72         weeks = int(temp[0])
73
74
75     if "hour" in course_length:
76         for i, t in enumerate(temp):
77             if "hour" in t:
78                 if "-" in temp[i - 1]:
79                     parts = temp[i - 1].split("-")
80                     hours = np.ceil((float(parts[0]) +
81                                     float(parts[1])) / 2).astype(
82                                     int)
83                 else:
84                     hours = np.ceil(float(temp[i - 1])
85                                     ).astype(int)
86
87     course_length = weeks * hours if weeks else hours
88
89
90     all_ClassCentral_courses.append({
91         "Title": title,
92         "Price_(in_$)": price,
93         "Difficulty": difficulty,
94         "Subject_Category": subject_category,
95         "Provider": provider,
96         "Course_Length_(in_Days)": course_length,
97         "Course_Language": course_language
98     })
99
100     df = pd.DataFrame(all_ClassCentral_courses)
101
102     return df

```

- **Coursera:** Η συλλογή βασίστηκε συγκεκριμένα σε αναζήτηση για μαθήματα πληροφορικής. Ο κώδικας εντοπίζει συγκεκριμένα CSS classes για να βρει τον τίτλο, τον πάροχο και τη διάρκεια των μαθημάτων κτλ.

```

1  def CourseraScraper(url):
2      Coursera_courses = []
3      number_of_courses = 6
4
5
6
7      res = requests.get(url)

```

```
8     soup = BeautifulSoup(res.text, 'html.parser')
9
10
11     blocks = soup.find_all("li", class_='cds-9_cds-11_cds-grid-item_cds-56_cds-64_cds-76_cds-89')
12
13     if not blocks:
14         print("No article content found.")
15         return pd.DataFrame()
16
17     random.shuffle(blocks)
18
19     for block in blocks:
20
21         if len(Coursera_courses) >= number_of_courses:
22             break
23
24         provider_tag = block.find("p", class_="cds-ProductCard-partnerNames")
25         provider = provider_tag.get_text(strip=True) if provider_tag else "N/A"
26
27         title_tag = block.find("h3", class_="cds-CommonCard-title")
28         title = title_tag.get_text(strip=True) if title_tag else "N/A"
29
30
31         price_tag=block.find("span", class_="css-1ast7yb")
32         price_tag2=price_tag.get_text(strip=True)
33         if "Free" in price_tag2:
34             price=0
35         else:
36             price=np.nan
37
38         index1=block.find("div", class_="cds-CommonCard-metadata")
39         index2=index1.get_text(strip=True)
40         index2=index2.split("A^")
41         difficulty=str(index2[0])
42         dur1=index2[2]
43         dur1=dur1.split("_")
44         num1=float(dur1[1])
45         num2=float(dur1[3])
46         mesos_oros=np.ceil((num2+num1)/2).astype(int)
47         dur2=dur1[4].lower()
48
49         if "week" in dur2:
50             course_length=mesos_oros*7
51         elif "month" in dur2:
52             course_length=mesos_oros*31
53         else:
54             course_length=np.nan
```



```

55
56         sub1=block.find("h3", class_="cds-CommonCard-title_css
57             -6ecy9b")
58         subject_category=sub1.get_text(strip=True)
59
60
61
62
63         a_tag = block.find("a", {"data-click-value": True})
64
65         if not a_tag:
66             continue
67
68         try:
69             props = json.loads(a_tag["data-click-value"])
70         except:
71             continue
72         course=str(props.get("filtersApplied"))
73         course=course.split("\'")
74         course_language=course[3]
75         subject_category=course[19]
76
77
78         Coursera_courses.append({
79             "Title": title,
80             "Price_(in_$)": price,
81             "Difficulty": difficulty,
82             "Subject_Category": subject_category,
83             "Provider": provider,
84             "Course_Length_(in_Days)": course_length,
85             "Course_Language": course_language
86         })
87
88         df = pd.DataFrame(Coursera_courses)
89         df = normalize_data(df)
90         return df

```

Πηγές API: Όπως ορίστηκε στο αρχείο `gui.py`, η εφαρμογή έχει σχεδιαστεί να επικοινωνεί και με 1 πηγή μέσω API

- **Udemy API:** Για την ανάκτηση πληροφοριών από τη βάση δεδομένων της Udemy.

Σε κάθε περίπτωση, το πρόγραμμα εμφανίζει στην κονσόλα τη διαδικασία συλλογής με μηνύματα όπως `[Source_Name] Status: Success` ή `Failed`, ώστε να ξέρουμε αν η λήψη έγινε σωστά.

2.2 Δομή και Αποθήκευση σε Αρχείο CSV

Μετά τη συλλογή και την κανονικοποίηση των δεδομένων, αυτά πρέπει να αποθηκευτούν τοπικά σε μόνιμη μορφή για μελλοντική χρήση. Λόγω της εκφώνησης, το αρχείο είναι τύπου CSV, το οποίο διαχειριζόμαστε μέσω της βιβλιοθήκης `pandas`.

Δομή των Δεδομένων (Metadata) Το αρχείο που παράγεται ονομάζεται `courses_1115515.csv`, το οποίο όπως ζητήτε περιλαμβάνει το ΑΜ ενός μέλους της ομάδας, όπως ορίζεται από την εκφώνηση της άσκησης. Κάθε εγγραφή στο αρχείο αντιπροσωπεύει ένα μάθημα και αποτελείται από 7 συγκεκριμένες στήλες (metadata) :

1. **Title:** Ο τίτλος του μαθήματος
2. **Price (in \$):** Το κόστος (0 αν είναι δωρεάν).
3. **Difficulty:** Το κανονικοποιημένο επίπεδο δυσκολίας
4. **Subject Category:** Η κανονικοποιημένη θεματική κατηγορία.
5. **Provider:** Ο πάροχος ή το πανεπιστήμιο που προσφέρει το συγκεκριμένο course.
6. **Course Length (in Days):** Η διάρκεια του μαθήματος υπολογισμένη σε ημέρες.
7. **Course Language:** Η γλώσσα διδασκαλίας.

Μηχανισμός Append Mode και Αποφυγή Διπλότυπων Η εφαρμογή παρέχει στον χρήστη τη δυνατότητα να επιλέξει αν θέλει να κάνει εγγραφή από την αρχή (Rewrite Mode) ή να προσθέσει τα νέα δεδομένα στο ήδη υπάρχον αρχείο χωρίς να χαθούν τα παλιά (Append Mode).

Για να αποφευχθεί η αποθήκευση διπλότυπων εγγραφών (duplicates) κατά τη διαδικασία του Append, υλοποιήθηκε η συνάρτηση `delete_duplicates_in_csv()` στο αρχείο `gui.py`. Η συνάρτηση αυτή διαβάζει το υπάρχον αρχείο CSV, απομονώνει τους τίτλους των αποθηκευμένων μαθημάτων σε ένα `set` και τους αφαιρεί από τα νέα δεδομένα που πρόκειται να εξαχθούν:

```

1
2  try:
3      pdd = pd.read_csv("courses_1115515.csv")
4      palioi = set(pdd["Title"])
5      neoi = set(Data_df["Title"])
6      pragmatika_neoi_titloi = neoi - palioi
7
8      new_data_only = Data_df[Data_df["Title"].isin(
9          pragmatika_neoi_titloi)]
10     return new_data_only
11 except FileNotFoundError:
12     return Data_df

```

Στη συνέχεια, η βασική συνάρτηση `export_data()` ελέγχει την επιλογή του χρήστη. Αν έχει επιλεγεί το Append Mode, αποθηκεύονται μόνο οι εγγραφές που επιστρέφει η παραπάνω συνάρτηση, θέτωντας το `headers` της `pandas` σε `"False"` για να μην καταστραφεί η δομή του αρχείου:

```

1  if AppendOrNot and mode == "a":
2      new_data = delete_duplicates_in_csv()
3      if new_data.empty:
4          messagebox.showinfo("CSV_Export_Info", "No_new_Data_were_
5              found!")
6          return
7      new_data.to_csv(filename, mode=mode, index=False, header=False
8          )
9  else:
10     Data_df.to_csv(filename, mode=mode, index=False, header=True)

```

Με την ολοκλήρωση της εξαγωγής, το σύστημα εμφανίζει επιβεβαιωτικό μήνυμα στην κονσόλα στη μορφή: `[System] Status: Export to courses_1115515.csv Successful`, εξασφαλίζοντας ότι όλα πήγαν καλά.

2.3 Μηχανισμός Κανονικοποίησης και Ομογενοποίησης Δεδομένων

Κατά τη συλλογή δεδομένων από τις πηγές μας, ένα από τα σημαντικότερα προβλήματα που προκύπτουν είναι η έλλειψη ομοιομορφίας στις τιμές των μεταβλητών. Κάθε πλατφόρμα χρησιμοποιεί τη δική της ορολογία για να περιγράψει παρόμοιες έννοιες (για παράδειγμα, το Harvard αναφέρει το εισαγωγικό επίπεδο ως `"introductory"`, ενώ το Coursera ως `"Beginner"`). Για την επίλυση αυτού του προβλήματος, σχεδιάστηκε και υλοποιήθηκε ένας μηχανισμός κανονικοποίησης μέσω της συνάρτησης `normalize_data()`.

Αντιστοίχιση Επιπέδων Δυσκολίας (Difficulty Mapping) Το σύστημα υποστηρίζει τρία συγκεκριμένα επίπεδα δυσκολίας: `beginner`, `intermediate` και `advanced`. Όλες οι εισερχόμενες τιμές μετατρέπονται αρχικά σε πεζά γράμματα (`.str.lower()`), αφαιρούνται τα περιττά κενά (`.str.strip()`) και στη συνέχεια αντιστοιχίζονται βάση συγκεκριμένων ορολογιών που έχουμε θεωρήσει:

Difficulty Mapping Rules

Τιμή Πηγής → Κανονικοποιημένη Τιμή

`"unknown" → "unknown"`

`"beginner", "introductory" → "beginner"`

`"intermediate", "mixed", "introductory, intermediate" → "intermediate"`

`"advanced", "introductory, intermediate, advanced" → "advanced"`

Κατηγοριοποίηση Θεματικών Ενοτήτων (Subject Category Mapping) Με τον ίδιο τρόπο, οι κατηγορίες των μαθημάτων ομογενοποιούνται σε συγκεκριμένους άξονες ενδιαφέροντος που υποστηρίζει το GUI, όπως `Computer Science`, `Business & Management` και `Humanities & Social Sciences` κτλ. Αυτό βοηθάει στην ομαδοποίηση διάφορων `courses` κάτω από μία γενική κατηγορία παρά μια συγκεκριμένη. Παρακάτω δίνονται μερικά για παράδειγμα:

Subject Category Mapping Rules

Κατηγορία Πηγής → Κανονικοποιημένη Θεματική Ενότητα

`"computer science", "programming", "python", "data science" → "Computer Science"`

`"business", "marketing" → "Business & Management"`

`"self improvement", "humanities", "personal development" → "Humanities & Social Sciences"`

Η υλοποίηση της παραπάνω λογικής βασίζεται στη βιβλιοθήκη `pandas`, η οποία επιτρέπει τη διανυσματική αντικατάσταση τιμών σε ολόκληρες τις στήλες του `DataFrame`, εξασφαλίζοντας υψηλή ταχύτητα εκτέλεσης:

Μικρό απόσπασμα από κώδικα(δεν είναι ολόκληρο):

```
1 mapping1 = {
```

```

2         'unknown': 'unknown',
3         'beginner': 'beginner',
4         'introductory': 'beginner',
5         'intermediate': 'intermediate',
6         'advanced': 'advanced',
7         'introductory, intermediate': 'intermediate',
8         'introductory, intermediate, advanced': 'intermediate'
9     }
10
11 mapping2 = {
12     'python': 'Computer_Science',
13     'finance & accounting': 'Business & Management',
14     'health & medicine': 'Health & Science',
15     'data science': 'Data_Science & STEM',
16     'teacher professional development': 'Humanities & Social_Sciences'
17     .....
18 }
19
20
21 df['Difficulty'] = df['Difficulty'].str.lower().str.strip().
    replace(mapping1)
22 df['Subject_Category'] = df['Subject_Category'].str.lower().str.
    strip()
23 df['Subject_Category'] = df['Subject_Category'].replace(mapping2)

```

Διαχείριση Ελλιπών (NaN) και Αριθμητικών Τιμών: Ο μηχανισμός κανονικοποίησης αναλαμβάνει επίσης τον καθαρισμό των αριθμητικών δεδομένων. Στην περίπτωση του κόστους (Price), εάν μια πηγή επιστρέψει κενή τιμή (NaN), επειδή το μάθημα είτε προσφέρεται δωρεάν είτε δεν δίνεται τιμή, αυτή αντικαθίσταται αυτόματα με την τιμή 0.

Επιπλέον, επειδή ορισμένες πηγές ενδέχεται να επιστρέψουν δεκαδικούς αριθμούς για τις τιμές ή τις διάρκειες, χρησιμοποιείται η συνάρτηση `np.ceil()` της βιβλιοθήκης `numpy` για τη στρογγυλοποίηση προς τα πάνω στον πλησιέστερο ακέραιο, και στη συνέχεια γίνεται μετατροπή του τύπου δεδομένων σε καθαρό ακέραιο (`astype(int)`):

```

1 df['Price_(in_$)'] = np.ceil(df['Price_(in_$)'].replace(np.nan, 0)
    ).astype(int)

```

Μέσω αυτής της διαδικασίας διασφαλίζεται ότι η βάση δεδομένων (.csv) διατηρεί απόλυτη ομοιογένεια, αποτρέποντας την εμφάνιση σφαλμάτων (Runtime Errors) κατά τη φάση του φιλτραρίσματος, της στατιστικής ανάλυσης και της παραγωγής γραφημάτων.

3. Σχεδιασμός και Υλοποίηση Γραφικού Περιβάλλοντος (GUI)

Για να μπορεί ο χρήστης να χειρίζεται εύκολα την εφαρμογή, φτιάξαμε ένα γραφικό περιβάλλον (GUI). Στόχος μας ήταν όλες οι λειτουργίες να βρίσκονται μαζεμένες σε ένα κεντρικό παράθυρο, ώστε να μην χρειάζεται καθόλου η χρήση του terminal για να τρέξουν οι διεργασίες.

3.1 Αρχιτεκτονική Διεπαφής με Tkinter

Το γραφικό περιβάλλον δημιουργήθηκε με τη βιβλιοθήκη `tkinter` της Python. Το παράθυρο έχει σταθερό μέγεθος 1280x720 pixels, ώστε να χωράνε άνετα όλα τα κουμπιά, οι λίστες επιλογών και το πλαίσιο που εμφανίζει τα μαθήματα.

Εξατομίκευση του Παραθύρου Όπως ζητήθηκε από την εκφώνηση της άσκησης, στον τίτλο του κεντρικού παραθύρου (`root.title`) έχουμε γράψει τα ονοματεπώνυμα και τους Αριθμούς Μητρώου (AM) των μελών της ομάδας μας.

Οργάνωση των Στοιχείων στο Παράθυρο Για να είναι τα στοιχεία τακτοποιημένα, χρησιμοποιήσαμε ένα κεντρικό Frame (`frame1`). Μέσα σε αυτό τοποθετήσαμε τα κουμπιά και τις λίστες το ένα κάτω από το άλλο (`side = "top"`), αφήνοντας μερικά κενά μεταξύ τους (`pady`) για να είναι το αποτέλεσμα πιο καθαρό στο μάτι:

Τα βασικά στοιχεία του GUI

- **Κουμπί «Προσθήκη μαθημάτων»:** Ξεκινάει τη συλλογή των δεδομένων ανάλογα με τη μέθοδο που έχει επιλεγεί.
- **Λίστα Επιλογής (OptionMenu-MethodMenu):** Επιτρέπει στον χρήστη να διαλέξει αν θέλει συλλογή μέσω Web Scraping ή μέσω API.
- **Κουμπιά Λειτουργιών:** Περιλαμβάνουν την «Εμφάνιση metadata», την «Επιλογή κριτηρίων» για τις συστάσεις, και την «Εμφάνιση γραφημάτων».
- **Κουμπί «Εξαγωγή δεδομένων σε CSV»:** Αποθηκεύει τοπικά τα μαθήματα στο αρχείο.
- **Πλαίσιο Εμφάνισης (Listbox):** Μια μεγάλη λίστα στο κάτω μέρος, με γραμματοσειρά Calibri (11), όπου τυπώνονται οι τίτλοι των μαθημάτων που μαζεύτηκαν στη μνήμη.

Αυτό είναι το κομμάτι του κώδικα από το αρχείο `gui.py` που δημιουργεί το παράθυρο, βάζει τον τίτλο με τα στοιχεία μας και ρυθμίζει τα πρώτα κουμπιά:

```

1  if __name__ == "__main__":
2      root = tk.Tk()
3      root.geometry("1280x720")
4
5      # Title with our Names and AM
6      root.title("Web_scraping_Python_Aggelopoulos_Grigoris_
              Panagiotis_AM:1115514_Kopitsas_Nikolaos_AM:1115515")
7

```

```

8      # Creating the Frame
9      frame1 = tk.Frame(root, width=500, height=500)
10     paddingYVal = 10
11     frame1.pack(pady=20)
12
13     # Button for adding data and dropdown menu for which method to
        use (API, WEB SCRAPING)
14     addSubjectsBtn = tk.Button(frame1, text="Add_Course", width
        =35, command=add_subjects)
15     addSubjectsBtn.pack(pady=paddingYVal, side="top")
16
17     methods = ["Web_Scraping", "API"]
18     method_opt = StringVar(value="Web_Scraping") # Default option
        for Web Scraping
19     methodMenu = OptionMenu(frame1, method_opt, *methods)
20     methodMenu.config(width=15)
21     methodMenu.pack(pady=5)
22
23     # Button to show the metadata
24     addSubjectsBtn = tk.Button(frame1, text="Show_Courses_Metadata
        ", width=35, command=readMetadata)
25     addSubjectsBtn.pack(pady=paddingYVal)
26     # Dropdown options
27     days = [""]
28     # Selected option variable
29     opt = StringVar(value="")
30     dropdownMenu = OptionMenu(frame1, opt, *days)
31     addSubjectsBtn = tk.Button(frame1, text="Select_Criteria",
        width=25, command=lambda: filtersWindow(days, dropdownMenu)
        )
32     addSubjectsBtn.pack(pady=paddingYVal)
33
34
35     # Button that shows the graphs (Bar, Line, Pie graphs)
36     showGraphsBtn = tk.Button(frame1, text="_Show_Graphs", width
        =35)
37     showGraphsBtn.pack(pady=paddingYVal)
38
39     exportsBtn = tk.Button(frame1, text="Extract_data_to_CSV",
        width=35, command=export_data)
40     exportsBtn.pack(pady=paddingYVal)
41
42
43     # Listbox to show the titles of the courses we add
44     listbox = tk.Listbox(frame1, width=100, font=("Calibri", 11))
45     listbox.pack(pady=20, padx=10)
46
47     root.mainloop()

```

Με αυτή τη δομή, το παράθυρο λειτουργεί γρήγορα και σωστά, ενώ συνδέσαμε κάθε κουμπί με την αντίστοιχη συνάρτηση στον κώδικα (μέσω της επιλογής `command`) ώστε η εφαρμογή να ανταποκρίνεται αμέσως μόλις ο χρήστης αλληλεπιδράσει.

3.2 Λειτουργίες Αναζήτησης και Φιλτραρίσματος

Για να μπορεί ο χρήστης να βρίσκει εύκολα τα μαθήματα που τον ενδιαφέρουν μέσα από τον μεγάλο όγκο data, φτιάξαμε ένα σύστημα φιλτραρίσματος. Η διαδικασία αυτή χωρίζεται σε δύο κομμάτια: το παράθυρο όπου ο χρήστης γράφει τι ψάχνει και τον έλεγχο των δεδομένων που γίνεται πίσω.

Το Παράθυρο Εισαγωγής Κριτηρίων (getUserInput.py) Όταν ο χρήστης πατάει το κουμπί «Επιλογή κριτηρίων» στο κεντρικό GUI, ανοίγει ένα νέο, μικρότερο παράθυρο με διαστάσεις 400x200 pixels. Εκεί υπάρχουν τέσσερα πεδία κειμένου όπου μπορεί να συμπληρώσει:

- Τη θεματική ενότητα που θέλει (π.χ. Computer Science).
- Το επίπεδο δυσκολίας (Beginner, Intermediate, Advanced).
- Το μέγιστο κόστος σε \$ (Price).
- Τη γλώσσα διδασκαλίας του μαθήματος.

Μόλις συμπληρώσει τα πεδία και πατήσει αποθήκευση, το πρόγραμμα μαζεύει αυτά τα δεδομένα-επιλογές και καλεί τη συνάρτηση `readFromCSVWithFilters()` για να ξεκινήσει ο έλεγχος.

Η Λογική του Φιλτραρίσματος (readFunctions.py) Το πρόγραμμα ανοίγει το αρχικό αρχείο δεδομένων (`courses_1115515.csv`) χρησιμοποιώντας τη βιβλιοθήκη `pandas`. Στη συνέχεια, ελέγχει ένα προς ένα όλα τα μαθήματα της λίστας για να δει αν ταιριάζουν με αυτά που ζήτησε ο χρήστης.

Για να είναι το φιλτράρισμα "έξυπνο" και να μην κολλάει η αναζήτηση, εφαρμόσαμε τους εξής κανόνες:

1. **Αγνόηση πεζών/κεφαλαίων:** Μετατρέπουμε τόσο τα κριτήρια όσο και τα δεδομένα σε πεζά γράμματα (`.lower()`) ώστε να μην έχει σημασία αν ο χρήστης έγραψε με κεφαλαία ή μικρά.
2. **Παράκαμψη κενών πεδίων:** Αν ο χρήστης αφήσει κάποιο πεδίο άδειο (string ''), το πρόγραμμα το αγνοεί και θεωρεί ότι το μάθημα περνάει επιτυχώς από το συγκεκριμένο φίλτρο.
3. **Έλεγχος τιμής:** Στο φίλτρο του κόστους ελέγχεται αν η τιμή του μαθήματος είναι μικρότερη ή ίση με το μέγιστο ποσό που έδωσε ο χρήστης.

Αποτέλεσμα Φιλτραρίσματος

- **Αν δεν βρεθεί κανένα μάθημα:** Εμφανίζεται ένα προειδοποιητικό μήνυμα λάθους (Error Message) που ενημερώνει τον χρήστη ότι δεν υπάρχουν αποτελέσματα με αυτά τα κριτήρια.
- **Αν βρεθούν μαθήματα:** Τα φιλτραρισμένα δεδομένα αποθηκεύονται σε ένα νέο αρχείο που ονομάζεται `filtered.csv` και αμέσως μετά καλείται αυτόματα ο αλγόριθμος για τον υπολογισμό των έξυπνων συστάσεων.

Παρακάτω φαίνεται το κομμάτι του κώδικα από το αρχείο `readFunctions.py` που κάνει αυτούς τους ελέγχους μέσα σε μια διπλή επανάληψη (`for loop`) για όλες τις γραμμές και τις στήλες του πίνακα:


```

1  for i in range(rows):
2      for j in range(cols):
3          #Check if difficulty matches or was left empty
4          diffComp = (df.values[i][2].lower() == difficulty.lower()
5                      or difficulty == '')
6          #Check if category matches or was left empty
7          categoryComp = (df.values[i][3].lower() == category.lower
8                          () or category == '')
9          #Check if the cost was left empty or which courses have
10         less cost than the max
11         maxCostComp = (maxCost == '' or df.values[i][1] <= float(
12             maxCost))
13         langComp = (df.values[i][6].lower() == language.lower() or
14                     language == '')
15         #if all match then we keep the line
16         if (diffComp and categoryComp and maxCostComp and langComp
17             ):
18             if(j==0):
19                 # need to update menu
20                 resDf.loc[resDf.shape[0]]= df.loc[i]
21
22 if(resDf.shape[0]<1):
23     tkinter.messagebox.showerror("Error", "No data found with
24     these criteria!")
25     return
26 else:
27     resDf.to_csv("filtered.csv", index=False)
28     calcCompositeScore() #we call the Composite Score Function

```

Με αυτόν τον τρόπο, ο χρήστης απομονώνει πολύ γρήγορα μόνο τα μαθήματα που τον ενδι-αφέρουν, δημιουργώντας μια καθαρή βάση δεδομένων έτοιμη για την παραγωγή των προτάσεων.

3.3 Ενσωμάτωση Μηνυμάτων Προόδου και Ελέγχου

Για να ξέρει ο χρήστης τι συμβαίνει κάθε στιγμή στην εφαρμογή και για να αποφεύγονται τα λάθη, χρησιμοποιήσαμε τα έτοιμα παράθυρα μηνυμάτων (messagebox) της βιβλιοθήκης tkinter. Αυτά τα παράθυρα εμφανίζονται όταν πρέπει να επιβεβαιωθεί μια ενέργεια, όταν κάτι πάει στραβά ή όταν μια διεργασία ολοκληρωθεί με επιτυχία.

Κατηγορίες Μηνυμάτων στον Κώδικα Στα αρχεία της εφαρμογής έχουμε βάλει τρία είδη τέτοιων μηνυμάτων:

- **Παράθυρο Επιβεβαίωσης (askyesno):** Μόλις ο χρήστης πατήσει το κουμπί «Προσθήκη μαθημάτων» στο αρχείο `gui.py`, εμφανίζεται ένα παράθυρο που τον ρωτάει αν είναι σίγουρος ότι θέλει να ξεκινήσει τη συλλογή με τη μέθοδο που διάλεξε (Web Scraping ή API). Έτσι, αν πατήσει το κουμπί κατά λάθος, μπορεί να ακυρώσει τη διαδικασία πριν αρχίσει το κατέβασμα.
- **Μηνύματα Επιτυχίας (showinfo):** Όταν το φιλτράρισμα των μαθημάτων τελειώσει σωστά στο αρχείο `readFunctions.py` και τα δεδομένα γραφτούν στο αρχείο, βγαίνει ένα ενημερωτικό μήνυμα που λέει «File saved successfully!».
- **Μηνύματα Λάθους (showerror):** Χρησιμοποιούνται για να σταματήσουν την εφαρμογή από το να κρασάρει αν κάτι δεν πάει καλά. Για παράδειγμα, αν το αρχείο CSV είναι άδειο ή

αν ο χρήστης βάλει κριτήρια που δεν αντιστοιχούν σε κανένα μάθημα, βγαίνει παράθυρο με τίτλο «Error» που εξηγεί τι φταίει (π.χ. «No data found with these criteria!»).

Γιατί είναι χρήσιμα αυτά τα μηνύματα

- **Έλεγχος λαθών:** Εμποδίζουν το πρόγραμμα να συνεχίσει να τρέχει με λάθος ή άδεια δεδομένα.
- **Ενημέρωση του χρήστη:** Ο χρήστης δεν αναρωτιέται αν κόλλησε η εφαρμογή, αφού βλέπει αμέσως αν η ενέργειά του πέτυχε ή απέτυχε.

Παρακάτω φαίνονται δύο χαρακτηριστικά παραδείγματα από τον κώδικα. Το πρώτο είναι η επιβεβαίωση πριν τη συλλογή στο `gui.py` και το δεύτερο είναι η εμφάνιση σφάλματος στο `readFunctions.py`:

```

1  # 1. Confirmation Window in gui.py
2  def add_subjects():
3      global Data_df, current_web_scraping_source_index
4      method = method_opt.get()
5
6      # Asks user before starting the web scraping or api
7      flag = askyesno("Confirm Data Collection", f"Are you sure you're ready to collect data with the method {method}?")
8      if not flag:
9          return # if he presses "No", then the function returns
10     .....
11
12
13
14  # 2. Error Window in readFunctions.py
15  if (resDf.shape[0] < 1):
16      #
17      ,
18      tkinter.messagebox.showerror("Error", "No data found with these criteria!")
19      return
20     .....

```

Με αυτά τα μηνύματα, η επικοινωνία του χρήστη με την εφαρμογή γίνεται πιο απλή και σίγουρη, αφού το ίδιο το πρόγραμμα τον καθοδηγεί και τον προστατεύει από λάθος χειρισμούς.

4. Στατιστική Επεξεργασία και Οπτικοποίηση

Σε αυτό το κεφάλαιο αναλύεται ο τρόπος με τον οποίο η εφαρμογή επεξεργάζεται τα συλλεχθέντα δεδομένα των μαθημάτων και τα μετατρέπει σε κατανοητά γραφήματα. Η στατιστική επεξεργασία επιτρέπει στον χρήστη να εντοπίσει εύκολα τάσεις, όπως τη σχέση μεταξύ της διάρκειας και του κόστους ενός μαθήματος, καθώς και την κατανομή των μαθημάτων ανάλογα με το επίπεδο δυσκολίας τους.

4.1 Ανάλυση Δεδομένων με Pandas

Η βιβλιοθήκη `pandas` αποτελεί τον πυρήνα της διαχείρισης δεδομένων στην εφαρμογή μας. Πριν από την παραγωγή οποιουδήποτε γραφήματος, τα δεδομένα που είναι αποθηκευμένα στο αρχείο `courses_1115515.csv` υφίστανται κατάλληλη επεξεργασία:

- **Ανάγνωση και Φόρτωση:** Τα δεδομένα φορτώνονται σε ένα `DataFrame` με τη χρήση της συνάρτησης `pd.read_csv()`, επιτρέποντας τη γρήγορη προσπέλαση των στηλών.
- **Ταξινόμηση (Sorting):** Για τις ανάγκες των γραφημάτων (όπως το Line Plot), τα δεδομένα ταξινομούνται με βάση τη διάρκεια των μαθημάτων σε ημέρες (`Course Length (in Days)`), χρησιμοποιώντας τη συνάρτηση `sort_values()`.
- **Φιλτράρισμα Κορυφαίων Αποτελεσμάτων:** Με τη μέθοδο `head(5)`, απομονώνονται τα 5 μαθήματα με τη μεγαλύτερη διάρκεια για την κατασκευή του Bar Chart και Line Plot.
- **Κατηγοριοποίηση και Καταμέτρηση:** Πραγματοποιείται έλεγχος και καταμέτρηση των εμφανίσεων κάθε επιπέδου δυσκολίας (*beginner, intermediate, advanced, unknown*) μέσα από δομές επανάληψης, ώστε να υπολογιστούν τα ακριβή ποσοστά για το Pie Chart.

4.2 Γραφική Απεικόνιση (Bar, Pie, Line Plot)

Για την οπτικοποίηση χρησιμοποιήθηκε η βιβλιοθήκη `matplotlib.pyplot`. Στο αρχείο `plots.py` έχουν υλοποιηθεί τρεις διαφορετικοί τύποι γραφημάτων, καθένας από τους οποίους εξυπηρετεί διαφορετικό σκοπό:

1. **Pie Chart:** Απεικονίζει την ποσοστιαία κατανομή των μαθημάτων ανάλογα με τη δυσκολία τους. Χρησιμοποιεί την ιδιότητα `explode` για να αναδείξει ελαφρώς την πρώτη κατηγορία, καθώς και σκιά (`shadow=True`) για καλύτερο οπτικό αποτέλεσμα.
2. **Bar Chart:** Εμφανίζει τα 5 μαθήματα με τη μεγαλύτερη διάρκεια. Οι ράβδοι βοηθούν στη καλύτερη σύγκριση των τίτλων των μαθημάτων ως προς τον χρόνο που απαιτείται για την ολοκλήρωσή τους. Επιπλέον η διάρκεια έγινε σε ημέρες λόγω ότι τα `courses` από `Web Scraping` δινόντουσαν σε ημέρες, εβδομάδες ή μήνες. Έτσι, επειδή το API τα έδινε σε ώρες έγινε η κατάλληλη μετατροπή σε ημέρες.
3. **Line Plot:** Παρουσιάζει τη σχέση μεταξύ της διάρκειας του μαθήματος (άξονας X) και της τιμής του (άξονας Y) για 5 μαθήματα με την μεγαλύτερη διάρκεια. Σχεδιάζεται με μαύρο χρώμα ενώ επίσης έχει bullets (`marker='o'`) σε κάθε σημείο δεδομένων με διαφορετικό χρώμα κι μέγεθος ώστε να ξεχωρίζονται αν τυχόν δυο μαθήματα έχουν ίδια στοιχεία και περιλαμβάνει πλέγμα (`grid(True)`) για ευκολία στην ανάγνωση τιμών. Επιπλέον υπάρχει παραθυράκι που αναφέρει κάθε χρώμα την τιμή κι διάρκεια για να είναι πιο κατανοητό ως προς τον χρήστη.

Όλα τα γραφήματα, εκτός από την προβολή τους στην οθόνη του χρήστη, αποθηκεύονται αυτόματα ως εικόνες (`.png`) τοπικά, ώστε να είναι διαθέσιμα για μελλοντική χρήση.

4.3 Ερμηνεία και Τάσεις Δεδομένων

Κατά την ανάπτυξη του συστήματος οπτικοποίησης και την ανάλυση των αποτελεσμάτων, προέκυψαν σημαντικές παρατηρήσεις καθώς και προβλήματα τα οποία λύθηκαν κατάλληλα:

Τεχνικές Προκλήσεις & Αντιμετώπιση τους

- **Διαχείριση Μηδενικών Δεδομένων στα Γραφήματα:** Μια βασική δυσκολία ήταν η εμφάνιση επιπέδων δυσκολίας που δεν είχαν καθόλου μαθήματα (είχαν τιμή 0). Επειδή αυτές οι κατηγορίες δεν έχουν μέγεθος για να σχεδιαστούν πάνω στο Pie Chart, η αυτόματη τοποθέτηση των ετικετών (labels) θα έκανε γράφημα δυσανάγνωστο. Για τον λόγο αυτό, έγινε πρόβλεψη στον κώδικα ώστε τα ονόματα των κατηγοριών μαζί με τα πλήθη τους να μην εμφανίζονται στον κύκλο αλλά να εμφανίζονται καθαρά σε ένα ξεχωριστό κουτί (legend) δίπλα από το γράφημα. Με αυτόν τον τρόπο, το διάγραμμα παραμένει καθαρό και ευανάγνωστο.
- **Διαχείριση Άγνωστης Δυσκολίας (Κατηγορία «Unknown»):** Τα μαθήματα που πήραμε από το API δεν είχαν καθόλου πληροφορία για τη δυσκολία τους. Για να μην κρασάρει το πρόγραμμα και για να είναι σωστά τα ποσοστά στο Pie Chart, προσθέσαμε μια έξτρα κατηγορία με όνομα «Unknown». Έτσι, το γράφημα δείχνει την αλήθεια για όλα τα δεδομένα που μαζέψαμε, χωρίς να αφήνει απέξω κανένα μάθημα.
- **Διόρθωση Περιθωρίων στο Bar Chart (Auto-layout):** Επειδή πολλοί τίτλοι μαθημάτων ήταν πολύ μεγάλοι, κόβονταν ή έπεφταν ο ένας πάνω στον άλλον στο Bar Chart. Το λύσαμε αυτό βάζοντας την εντολή `figure.autolayout: True` στον κώδικα. Αυτή η ρύθμιση μεγαλώνει αυτόματα τα περιθώρια του γραφήματος, ώστε να χωράνε και να διαβάζονται καθαρά όλοι οι τίτλοι των μαθημάτων.

Παρακάτω παρατίθεται το σχετικό απόσπασμα κώδικα από το αρχείο `plots.py`, όπου φαίνεται η διαχείριση των παραπάνω παραμέτρων:

```

1  def LinePlot():
2      df = pd.read_csv("courses_1115515.csv", delimiter=',')
3      df = df.sort_values(by="Course_Length_(in_Days)", ascending=
4          False)
5      df = df.head(5)
6
7      df = df.sort_values(by="Course_Length_(in_Days)", ascending=
8          True)
9
10     x = df["Course_Length_(in_Days)"]
11     y = df["Price_(in_$)"]
12     plt.plot(x, y, color="black", zorder=1)
13
14     chromata = ['blue', 'green', 'orange', 'purple', 'red']
15
16     plt.plot(x.iloc[0], y.iloc[0], marker='o', color=chromata[0],
17         markersize=10,
18         label=f"Price:_{y.iloc[0]}$, _Days:_{x.iloc[0]}")
19     plt.plot(x.iloc[1], y.iloc[1], marker='o', color=chromata[1],
20         markersize=8,
```

```

17         label=f"Price:_{y.iloc[1]}$,_{Days:_{x.iloc[1]}}"
18     plt.plot(x.iloc[2], y.iloc[2], marker='o', color=chromata[2],
19             markersize=6,
20             label=f"Price:_{y.iloc[2]}$,_{Days:_{x.iloc[2]}}"
21     plt.plot(x.iloc[3], y.iloc[3], marker='o', color=chromata[3],
22             markersize=4,
23             label=f"Price:_{y.iloc[3]}$,_{Days:_{x.iloc[3]}}"
24     plt.plot(x.iloc[4], y.iloc[4], marker='o', color=chromata[4],
25             markersize=2,
26             label=f"Price:_{y.iloc[4]}$,_{Days:_{x.iloc[4]}}"
27
28     plt.grid(True)
29     plt.xlabel("Course_Length_(in_Days)")
30     plt.ylabel("Price_(in_$)")
31     plt.title("Line_Plot_with_Grid")
32
33     # Legend with time and price
34     plt.legend(loc="upper_center")
35
36     plt.savefig("LinePlot.png")
37     plt.show()
38
39 def PieChart():
40     df = pd.read_csv("courses_1115515.csv", delimiter=',')
41     diffCol = df.get("Difficulty")
42
43     data = [0, 0, 0, 0]
44     for col in diffCol:
45         if col == "beginner":
46             data[0] += 1
47         if col == "intermediate":
48             data[1] += 1
49         if col == "advanced":
50             data[2] += 1
51         if col == "unknown":
52             data[3] += 1
53
54     label1 = "Beginner:_" + str(data[0])
55     label2 = "Intermediate:_" + str(data[1])
56     label3 = "Advanced:_" + str(data[2])
57     label4 = "Unknown:_" + str(data[3])
58
59     my_labels = [label1, label2, label3, label4]
60     plt.figure(figsize=(10, 7))
61     plt.title("Courses_based_on_difficulty")
62     myexplode = [0.1, 0, 0, 0]
63     plt.pie(data, explode=myexplode, shadow=True)
64
65     plt.legend(title="Difficulties", labels=my_labels)
66     plt.savefig("PieChart.png")
67     plt.show()

```

```

67 def BarChart():
68     #making sure that titles wont be cut
69     matplotlib.rcParams.update({'figure.autolayout': True})
70     df = pd.read_csv("courses_1115515.csv", delimiter=',')
71     dfSorted= df.sort_values(by= "Course_Length_(in_Days)" ,
72                             ascending=False)
73
74     top5=dfSorted.head(5)
75
76     names = top5["Title"]
77     values = top5["Course_Length_(in_Days)"]
78     cols=len(names)
79
80
81     fig = plt.figure(figsize=(15, 8))
82     bars=plt.bar(range(cols),values, width=0.4, color="#4CAF50")
83     plt.xticks(range(cols), names, rotation=90)
84     plt.bar_label(bars)
85
86
87     #We used duration in days not in hours cause all courses from
88         web scraping had the duration either in days, weeks or
89         months
90     plt.ylabel('Duration_(in_Days)', fontsize = 12.5)
91     plt.tick_params(axis='x',rotation=90,labels=12.5)
92     plt.tick_params(axis='y', labels=12.5)
93     plt.savefig("BarChart.png")
94     plt.show()

```

Η προσέγγιση αυτή προσφέρει στον χρήστη μια ολοκληρωμένη και αξιόπιστη εικόνα των δεδομένων από την διαδικασία συλλογής από εξωτερικές πηγές (Scraping/API).

5. Μηχανή Έξυπνων Συστάσεων (Recommendation Engine)

Σε αυτό το κεφάλαιο θα δούμε πώς φτιάξαμε και πώς λειτουργεί το σύστημα συστάσεων της εφαρμογής μας. Η διαδικασία ξεκινάει από το κεντρικό παράθυρο στο αρχείο `gui.py`, όπου ο χρήστης μπορεί να ανοίξει το παράθυρο των φίλτρων για να αναζητήσει μαθήματα.

5.1 Λογική και Σχεδιασμός του Αλγορίθμου Κατάταξης

Για τα φίλτρα χρησιμοποιούμε από το αρχείο `applyFilters.py` την συνάρτηση `filtersWindow()`, η οποία ανοίγει ένα απλό γραφικό παράθυρο για να βάλει ο χρήστης τα κριτήριά του. Η επεξεργασία αυτών των κριτηρίων εκτελείται από τη συνάρτηση `readFromCSVWithFilters()` του αρχείου `readFunctions.py`.

Όταν σχεδιάζαμε τα φίλτρα, σκεφτήκαμε και τις εξής περιπτώσεις:

- Αν ο χρήστης αφήσει κάποιο πεδίο άδειο (""), το πρόγραμμα το δέχεται αυτόματα ως True. Αυτό σημαίνει ότι το συγκεκριμένο κριτήριο δεν φιλτράρει τα δεδομένα και προχωράμε παρακάτω.

- Στα πεδία για την κατηγορία, τη δυσκολία και τη γλώσσα, μετατρέπουμε όλα τα κείμενα σε μικρά γράμματα με τη `lower()`. Έτσι, δεν έχει σημασία αν ο χρήστης έγραψε με κεφαλαία ή μικρά, η σύγκριση γίνεται σωστά.
- Στο πεδίο με την κατηγορία επιπλέον εφαρμόστηκε η συνάρτηση `replace(" ", ",")` η οποία αφαιρεί τα κενά ώστε να μην επηρεάσουν τα αποτελέσματα.
- Στο φίλτρο για το μέγιστο κόστος, κρατάμε μόνο τα μαθήματα που έχουν τιμή μικρότερη ή ίση από αυτή που ζήτησε ο χρήστης.
- Όσα μαθήματα περάσουν όλα τα φίλτρα, αποθηκεύονται στο αρχείο `filtered.csv`.

Η βασική δυσκολία που βρήκαμε εδώ ήταν να μειώσουμε και να απλοποιήσουμε τους ελέγχους, ώστε ο κώδικας να μην είναι τεράστιος και να τρέχει γρήγορα.

5.2 Υπολογισμός Σύνθετης Βαθμολογίας (Composite Score)

Αφού ετοιμαστεί το αρχείο με τα φιλτραρισμένα μαθήματα, τρέχει ο αλγόριθμος για τις έξυπνες συστάσεις από τη συνάρτηση `calcCompositeScore()` του αρχείου `compositeScore.py`. Σκοπός μας είναι να βαθμολογήσουμε τα μαθήματα, με το ιδανικό αποτέλεσμα να παίρνει την ανώτατη βαθμολογία, δηλαδή σκορ 1.

Το τελικό σκορ προκύπτει συνδυάζοντας το κόστος και τη διάρκεια, χρησιμοποιώντας τα εξής βάρη:

- **Κόστος (60%):** Δώσαμε μεγαλύτερη βαρύτητα στην τιμή (συντελεστής 0.6) επειδή αυτή η πληροφορία υπάρχει πάντα στις πηγές μας. Όσο πιο μικρό είναι το κόστος, τόσο το καλύτερο για το σκορ.
- **Διάρκεια (40%):** Η διάρκεια έχει λίγο μικρότερη βαρύτητα (συντελεστής 0.4) επειδή μερικές φορές λείπει από τα δεδομένα και μας έρχεται ως 0. Όσο μικρότερη είναι η διάρκεια, τόσο το καλύτερο.

Οι μαθηματικοί τύποι που χρησιμοποιούνται βασίζονται στην κανονικοποίηση Min-Max για να φέρουμε τις τιμές στο ίδιο εύρος (από 0 έως 1) και να μπορούμε να τις συγκρίνουμε. Θεωρώντας ότι οι μέγιστες και οι ελάχιστες τιμές διαφέρουν, έχουμε τους εξής τύπους:

$$normalizedCost = \left(1 - \frac{Cost - \min(Cost)}{\max(Cost) - \min(Cost)}\right) \times 0.6$$

$$normalizedDuration = \left(1 - \frac{Duration - \min(Duration)}{\max(Duration) - \min(Duration)}\right) \times 0.4$$

Το τελικό **Composite Score** βγαίνει προσθέτοντας αυτές τις δύο τιμές και στρογγυλοποιείται στα 2 δεκαδικά ψηφία:

$$CompositeScore = normalizedCost + normalizedDuration$$

Εδώ πρέπει να αναφερθεί ότι οι πράξεις για εύρεση μέγιστου, ελάχιστου, πρόσθεση κι στρογγυλοποίηση έγιναν πάνω σε μία σειρά με όνομα `Cost` κι οι πράξεις υλοποιήθηκαν μέσω της βιβλιοθήκης `numpy` κι των συναρτήσεων `np.max()`, `np.min()`, `np.add()`, `np.round()`.

5.3 Διαχείριση Ελλιπών Στοιχείων και Παρουσίαση

Η κυριότερη πρόκληση για εμάς ήταν να καταλάβουμε πώς θα δουλέψει ο αλγόριθμος στην πράξη και να βρούμε, κάνοντας δοκιμές (τεστάρισμα), τα σημεία που μπορεί να κολλήσουν το πρόγραμμα. Για παράδειγμα, είδαμε ότι αν στα φίλτρα βρεθεί μόνο ένα μάθημα, το πρόγραμμα χωρίς τον κατάλληλο χειρισμό θα κράσαρε αμέσως γιατί θα προσπαθούσε να κάνει διαίρεση με το μηδέν.

Για να λύσουμε αυτά τα προβλήματα, βάλουμε κάποιους ελέγχους μέσα στο αρχείο `compositeScore.py`:

- **Προστασία από κρασάρισμα:** Αν η αφαίρεση του ελαχίστου από το μέγιστο μας κάνει μηδέν (επειδή βρέθηκε μόνο ένα μάθημα ή όλα έχουν την ίδια ακριβώς τιμή ή διάρκεια), ο κώδικας αντί να κάνει την διαίρεση, δίνει αυτόματα μια έτοιμη μέση βαθμολογία, δηλαδή 0.6 για το κόστος και 0.4 για τη διάρκεια.
- **Διαχείριση ελλιπούς διάρκειας:** Αν η διάρκεια ενός μαθήματος είναι 0, σημαίνει ότι λείπει αυτή η πληροφορία από το API ή το Web Scraping, αφού είναι αδύνατο ένα μάθημα να μην κρατάει καθόλου. Σε αυτή την περίπτωση, ο αλγόριθμος δεν αντιστρέφει την κλίμακα (δεν κάνει την πράξη 1-τιμή), αλλά προσθέτει 0, για να μην πάρει άδικα υψηλό σκορ ένα μάθημα που δεν έχει γεμάτα στοιχεία.

Παρουσίαση των Top 3 Μαθημάτων: Αφού υπολογιστούν όλα τα σκορ, ταξινομούμε τα μαθήματα στο αρχείο `filtered.csv` σε φθίνουσα σειρά μέσω της συνάρτησης `sort_values()` της βιβλιοθήκης `pandas` με παράμετρο `ascending=False`, δηλαδή από το μεγαλύτερο σκορ προς το μικρότερο. Στο τέλος, η συνάρτηση `readTop3Subjects()` από το αρχείο `readFunctions.py` διαβάζει τα αποτελέσματα και ανοίγει ένα παράθυρο με μπάρα για κύλιση (Scrollable Frame), δείχνοντας στον χρήστη μόνο τα **Top 3** καλύτερα μαθήματα που ταιριάζουν σε αυτό που έψαχνε.

6. Τεκμηρίωση Υλοποίησης (Screenshots & Λύσεις)

Σε αυτό το κεφάλαιο αναλύουμε τις βασικές αποφάσεις που πήραμε όταν σχεδιάζαμε την εφαρμογή, τα προβλήματα που μας παρουσιάστηκαν στην πορεία και τις λύσεις που δώσαμε στον κώδικα για να δουλεύουν όλα σωστά.

6.1 Παραδοχές Σχεδιασμού

Όταν ξεκινήσαμε να φτιάχνουμε το πρόγραμμα, συμφωνήσαμε σε μερικά βασικά πράγματα για το πώς θα διαχειριζόμαστε τα δεδομένα:

- **Κοινό Αρχείο Αποθήκευσης:** Όλα τα μαθήματα που μαζεύουμε, είτε με Web Scraping από το Harvard, το Class Central και το Coursera, είτε μέσω API από το Udemy Free, αποθηκεύονται μαζί σε ένα κοινό αρχείο CSV με όνομα `courses_1115515.csv`.
- **Μετατροπή της Διάρκειας σε Ημέρες:** Επειδή κάθε site έδινε τον χρόνο με τον δικό του τρόπο (σε ώρες, εβδομάδες ή μήνες), αποφασίσαμε να τα μετατρέπουμε όλα σε ημέρες για να υπάρχει μια κοινή λογική. Στο API της Udemy που μας έρχονταν ώρες, τις διαιρούμε με το 24, ενώ στο Harvard πολλαπλασιάζουμε τις εβδομάδες με το 7. Αν κάπου δεν υπήρχε καθόλου χρόνος, τον ορίζουμε ως 0.
- **Τιμή για τα Δωρεάν Μαθήματα:** Στις πλατφόρμες που έχουν δωρεάν μαθήματα (όπως το Class Central ή το Udemy), αν η τιμή ήταν κενή (NaN), τη μετατρέπουμε αυτόματα σε 0.

- **Ομαδοποίηση Κατηγοριών και Δυσκολίας:** Επειδή κάθε πλατφόρμα χρησιμοποιούσε διαφορετικές λέξεις για τα ίδια πράγματα (π.χ. άλλος έγραφε "introductory" και άλλος "beginner"), φτιάξαμε σταθερά λεξικά στον κώδικα (mappings). Έτσι, μαζέψαμε όλες τις δυσκολίες σε 4 συγκεκριμένα επίπεδα και τις θεματικές ενότητες σε 5 μεγάλες κατηγορίες.

6.2 Προβλήματα που Αντιμετωπίστηκαν και Λύσεις

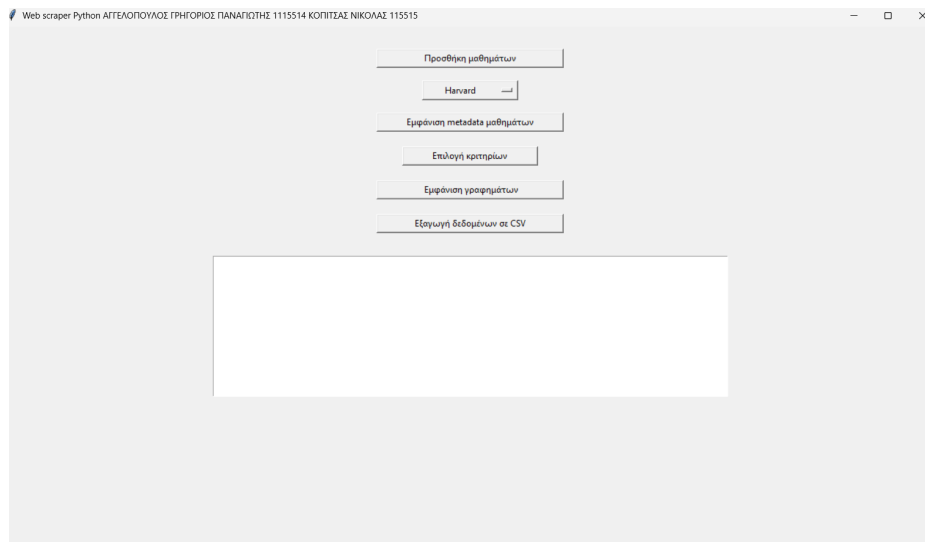
Κάνοντας δοκιμές στο πρόγραμμα, μας έβγαλαν κάποια bugs και δυσκολίες, τα οποία λύσαμε με τους εξής τρόπους:

- **Κράσαρε ο αλγόριθμος συστάσεων με τη διαίρεση με το μηδέν:** *Πρόβλημα:* Αν ο χρήστης έβαζε πολύ αυστηρά φίλτρα και έμενε μόνο ένα μάθημα στο αρχείο `filtered.csv` (ή αν όλα τα μαθήματα είχαν την ίδια ακριβώς τιμή), ο τύπος Min-Max προσπαθούσε να διαιρέσει με τη διαφορά `max - min`, η οποία ήταν 0, και το πρόγραμμα έσπαγε.
Λύση: Στο αρχείο `compositeScore.py` βάλαμε ελέγχους `if` πριν από τις πράξεις. Αν η διαφορά είναι 0, ο κώδικας σταματάει τη διαίρεση και δίνει αυτόματα έτοιμα σκορ (0.6 για το κόστος και 0.4 για τη διάρκεια).
- **Άδικα υψηλό σκορ σε μαθήματα χωρίς διάρκεια:** *Πρόβλημα:* Επειδή ο αλγόριθμος αναποδογυρίζει την κλίμακα της διάρκειας (ώστε οι λιγότερες μέρες να παίρνουν μεγαλύτερο βαθμό), αν ένα μάθημα είχε διάρκεια 0 επειδή έλειπε η πληροφορία από το site, η πράξη `1 - duration` το έκανε 1, και έπαιρνε άδικα την καλύτερη βαθμολογία.
Λύση: Βάλαμε μια συνθήκη που ελέγχει αν η διάρκεια είναι 0. Αν είναι, ο κώδικας προσπερνάει την αναστροφή και το μάθημα παίρνει μηδέν πόντους από τη διάρκεια, ώστε να μην ευνοηθεί χωρίς λόγο.
- **Προβλήματα με τα γραφήματα της Matplotlib:** *Πρόβλημα:* Στο Pie Chart, τα μαθήματα από το API δεν είχαν δυσκολία και χάλνουςαν τα ποσοστά. Επίσης, στο Bar Chart, οι τίτλοι των μαθημάτων ήταν μεγάλοι και έπεφταν ο ένας πάνω στον άλλο στον οριζόντιο άξονα.
Λύση: Στο Pie Chart προσθέσαμε μια κατηγορία "Unknown" για να μπαίνουν εκεί όσα δεν έχουν επίπεδο δυσκολίας. Στο Bar Chart χρησιμοποιήσαμε την εντολή `rotation=90` για να γυρίσουν τα ονόματα κάθετα, και ενεργοποιήσαμε το `autolayout` για να χωράνε σωστά τα κείμενα.
- **Το UI δεν χωρούσε τα πολλά δεδομένα:** *Πρόβλημα:* Όταν θέλαμε να δείξουμε τα metadata ή τα φιλτραρισμένα μαθήματα στο Tkinter, ο πίνακας ήταν τεράστιος και έβγαине έξω από την οθόνη.
Λύση: Ψάξαμε online για μία έτοιμη συνάρτηση όπου δημιουργεί ένα scroll bar κι στη συνέχεια την τροποποιήσαμε στις δικές μας ανάγκες, φτιάχνοντας έτσι την συνάρτηση `ScrollableFrame` στο αρχείο `scrollableFrame.py`. Αυτή συνδυάζει το Canvas του Tkinter με κάθετο κι οριζόντιο scroll bar, οπότε ο χρήστης μπορεί να σκρολάρει άνετα και να τα βλέπει όλα.

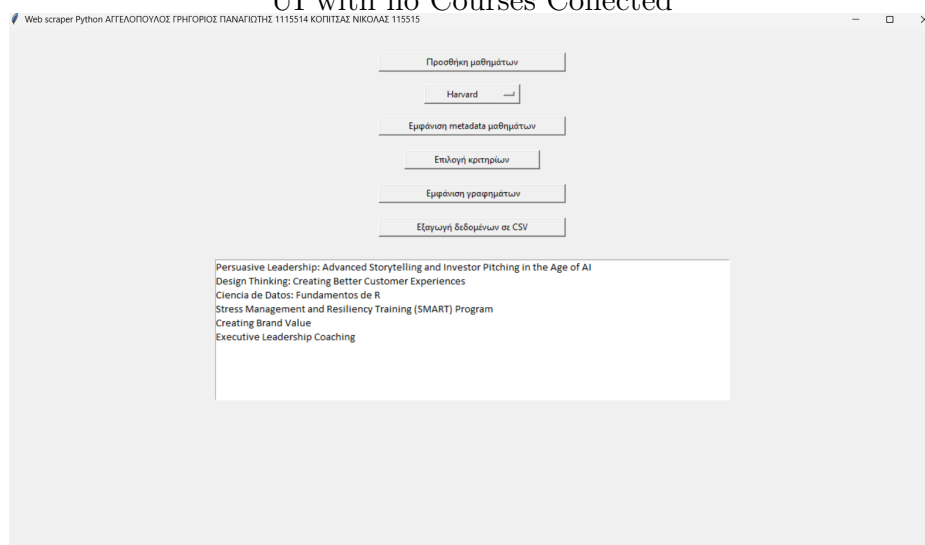
6.3 Στιγμιότυπα Εκτέλεσης (Screenshots)

Παρακάτω έχουμε βάλει τα στιγμιότυπα από τη λειτουργία της εφαρμογής μας, όπως φαίνονται όταν τρέχουμε το κεντρικό αρχείο `gui.py`:

1. **Κεντρικό Παράθυρο:** Η αρχική οθόνη του προγράμματος με όλα τα κουμπιά για τη συλλογή δεδομένων, τα φίλτρα, τα γραφήματα και την εξαγωγή.

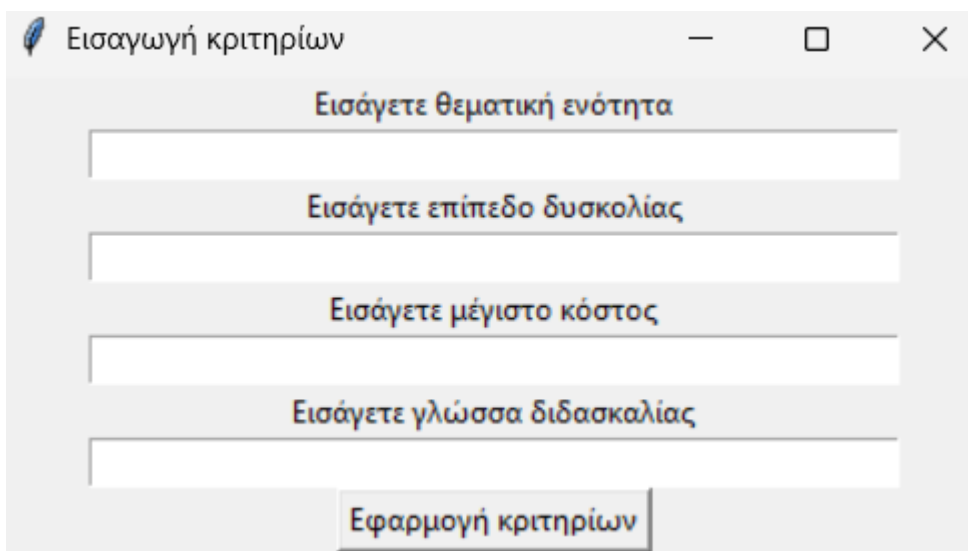


UI with no Courses Collected



UI with Courses Collected

2. **Παράθυρο Φίλτρων:** Το μικρό παράθυρο από το αρχείο `applyFilters.py` όπου ο χρήστης πληκτρολογεί τι ψάχνει.



Filter Window

3. **Πίνακας Metadata:** Η εμφάνιση όλων των μαθημάτων του αρχείου `courses_1115515.csv`

μέσα στο scrollable frame.

Title	Price (in \$)	Difficulty	Subject Category	Provider	Course Language	Course Length (in Days)
Treating Obesity: The Blackburn Course in Obesity Medicine	1295	advanced	health & science	Harvard Medical School	English	0
In-Place Filter Testing Workshop	3000	advanced	health & science	Harvard T.H. Chan School of Public Health	English	0
High Potentials Leadership Program	18500	advanced	business & management	Harvard Business School	English	7
Executive Leadership Coaching	3200	intermediate	business & management	Harvard DCE Professional & Executive Development	English	0
AI for Leaders	1850	intermediate	business & management	Harvard Business School	English	0

Metadata Window

4. **Παρουσίαση Top 3:** Το παράθυρο που ανοίγει και δείχνει μόνο τις 3 καλύτερες προτάσεις μαθημάτων σύμφωνα με το Composite Score. Για να γίνει πιο κατανοητό έχουμε εφαρμόσει φίλτρα και εμφανίζονται τα παρακάτω αποτελέσματα.

Εισαγωγή κριτηρίων

Εισάγετε θεματική ενότητα

Computer Science

Εισάγετε επίπεδο δυσκολίας

beginner

Εισάγετε μέγιστο κόστος

300

Εισάγετε γλώσσα διδασκαλίας

English

Εφαρμογή κριτηρίων

Filters

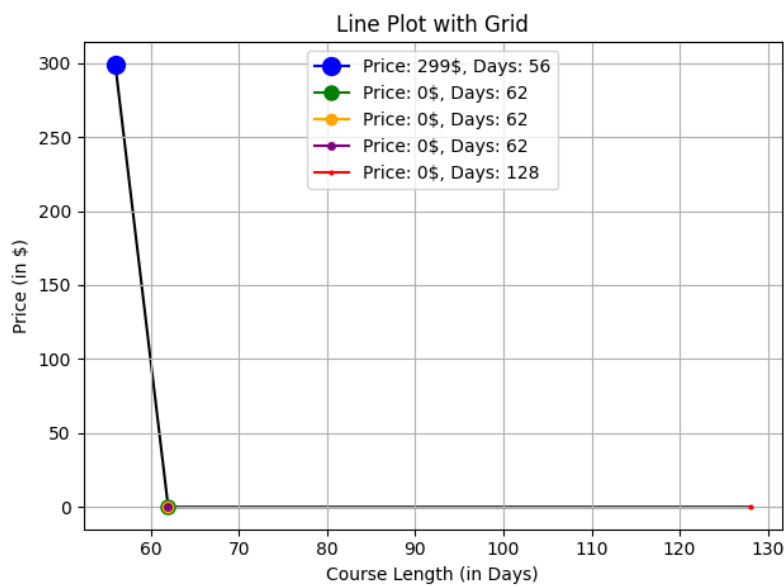
Title	Price (in \$)	Difficulty	Subject Category	Provider	Course Language	Course Length (in Days)
HTML and CSS in depth	0	beginner	Computer Science	Coursera	English	21
Introduction to Web Development	0	beginner	Computer Science	Coursera	English	21
Introduction to HTML, CSS, & JavaScript	0	beginner	Computer Science	Coursera	English	21

Top 3 Courses

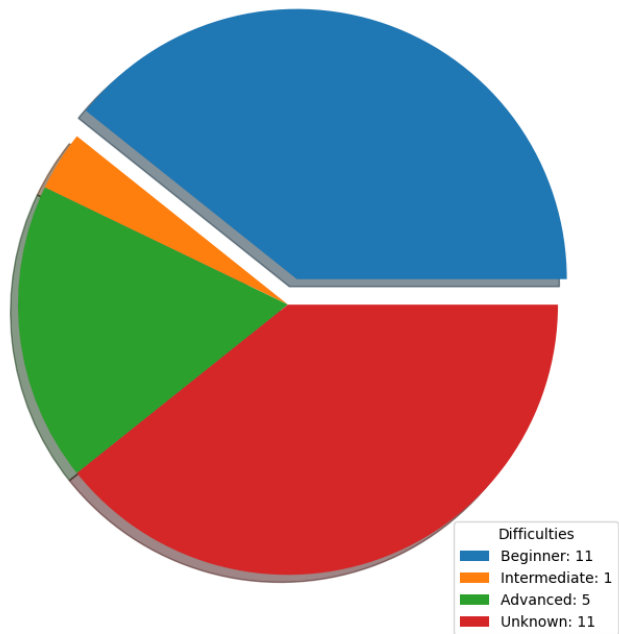
0.98
0.98
0.98

Top 3 Course's Composite Score

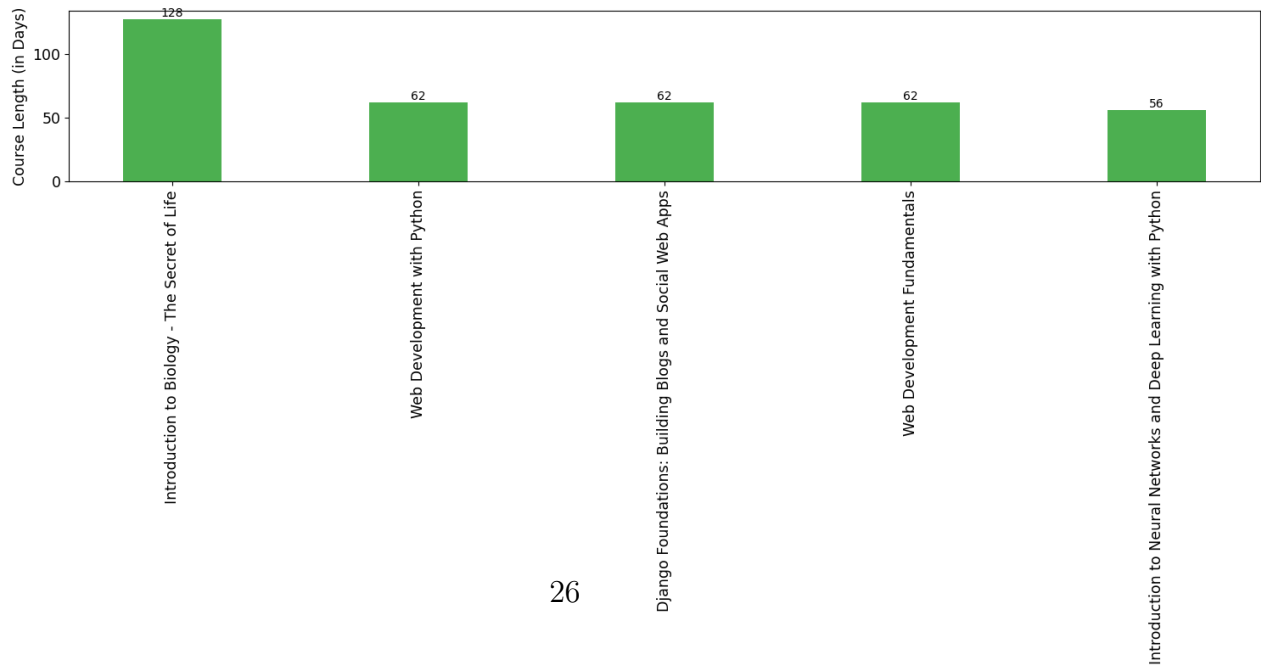
5. **Στατιστικά Γραφήματα:** Οι τρεις εικόνες που αποθηκεύονται αυτόματα (LinePlot.png, PieChart.png, BarChart.png) από τις συναρτήσεις της matplotlib.



Line Plot
Courses based on difficulty



Pie Chart



Παρατήρηση όσον αφορά το Line Plot: Με βάση την οπτική ανάλυση του γραφήματος `LinePlot.png`, παρατηρούμε ότι η αύξηση της χρονικής διάρκειας των μαθημάτων δεν συνεπάγεται αναλογική αύξηση του κόστους τους, καθώς στο συγκεκριμένο δείγμα η τάση εμφανίζεται αντίστροφη. Συγκεκριμένα, το μάθημα με τη μικρότερη διάρκεια από τα πέντε (56 ημέρες) είναι το μόνο με υψηλό κόστος (\$299), ενώ όσο η διάρκεια αυξάνεται στις 62 ημέρες (όπου συμπίπτουν 3 μαθήματα – πράσινος, πορτοκαλί, μωβ) και στις 128 ημέρες (κόκκινο), η τιμή μηδενίζεται.

Αυτό δείχνει ότι στο τρέχον δείγμα τα πολύ μεγάλα σε διάρκεια μαθήματα προσφέρονται δωρεάν. Ωστόσο, η τάση αυτή ισχύει αποκλειστικά για το συγκεκριμένο σύνολο δεδομένων και δεν αποτελεί γενικό κανόνα, καθώς σε ένα διαφορετικό δείγμα η αύξηση της διάρκειας θα μπορούσε να συνεπάγεται αναλογική αύξηση του κόστους λόγω των περισσότερων ωρών διδασκαλίας.